

✓ Assignment-6

Aim: Implementing Web Scraping in Python with BeautifulSoup.

Theory: Web scraping involves automating the extraction of data from websites. Python offers powerful libraries like `requests` for fetching web page content and `BeautifulSoup` (often imported as `bs4`) for parsing its HTML structure. The `requests` library handles the communication with web servers, retrieving the raw HTML, while `BeautifulSoup` enables navigation and searching within this HTML to pinpoint and extract desired data elements.

In this assignment, our focus was on scraping tabular data from a specific webpage. To ensure successful retrieval and avoid common bot detection mechanisms, we configured custom HTTP headers, such as the `User-Agent`, to mimic a standard web browser. Once the content was secured, `BeautifulSoup` was employed to systematically parse the HTML. This involved using specific HTML tags, class names, and ID attributes to precisely locate the data embedded within tables.

Particular attention was given to making the scraping process robust. This included implementing conditional checks to gracefully handle instances of missing or malformed HTML tags. Such measures prevent program crashes and ensure that as much available data as possible is extracted, even from imperfect web structures.

Post-extraction, the data—especially that from tables—was structured and formatted. The final step involved exporting this cleaned and organized data into a CSV file, making it readily accessible for subsequent analysis. Libraries like Pandas can then be utilized for further data manipulation and validation.

This practical exercise covered the following key aspects:

1. Extracting tabular information from a web page.
2. Saving the collected data into a CSV file format.
3. Developing resilient code to manage absent or broken HTML elements.
4. Targeting and scraping data using HTML class and ID attributes.
5. Utilizing the `requests` library with custom headers for web access.

```
!pip install beautifulSoup
import requests
from bs4 import BeautifulSoup
import csv
import pandas as pd

print("Libraries imported successfully.")
```

Collecting BeautifulSoup

Downloading BeautifulSoup-3.2.2.tar.gz (32 kB)

error: subprocess-exited-with-error

× python setup.py egg_info did not run successfully.

exit code: 1

↳ See above for output.

note: This error originates from a subprocess, and is likely not a problem

Preparing metadata (setup.py) ... error

error: metadata-generation-failed

× Encountered error while generating package metadata.

↳ See above for output.

note: This is an issue with the package mentioned above, not pip.

hint: See above for details.

Libraries imported successfully.

```
headers = {
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537
}
url = "https://en.wikipedia.org/wiki/List_of_brightest_stars"

def run_assignment_6():
    response = requests.get(url, headers=headers)
    soup = BeautifulSoup(response.content, 'html.parser')
    table = soup.find('table', class_='wikitable')

    scraped_rows = []

    if table:
        rows = table.find_all('tr')

        for row in rows:
            cells = row.find_all(['th', 'td'])
            cleaned_row = []
            for cell in cells:
                if cell:
                    cleaned_row.append(cell.get_text(strip=True))
                else:
                    cleaned_row.append("N/A")

            if cleaned_row:
                scraped_rows.append(cleaned_row)

    with open('stars_data.csv', 'w', newline='', encoding='utf-8') as f:
        writer = csv.writer(f)
        writer.writerows(scraped_rows)

    print(f"Scrape Complete: {len(scraped_rows)} rows saved to stars_dat
else:
    print("Table target not found. Check the class attribute or URL.")
```

```

run_assignment_6()

try:
    df = pd.read_csv('stars_data.csv')
    print("Verification: First 10 rows of the stored data:")
    display(df.head(10))
except Exception as e:
    print(f"Verification Failed: {e}")

```

Scrape Complete: 8 rows saved to stars_data.csv
Verification: First 10 rows of the stored data:

Wolf-Rayet star	
0	O-type star
1	B-type star
2	A-type star
3	F-type star
4	G-type star
5	K-type star
6	M-type star

Conclusion:

This assignment successfully showcased the methodology of web scraping using Python. We effectively retrieved, processed, and stored web-based data, specifically focusing on information presented in tables, by leveraging the `requests` and `BeautifulSoup` libraries.

The strategic inclusion of HTTP headers proved crucial for reliable webpage access. Furthermore, the implementation of safeguards for handling irregular HTML structures contributed significantly to the program's stability. By utilizing specific class and ID attributes, the data extraction process was both precise and highly efficient, culminating in a well-structured dataset ready for further use.

